CSCI3180 Principles of Programming Languages

Ryan Chan

May 1, 2026

Abstract

This is a note for **CSCI3180 Principles of Programming Languages**.
Contents are adapted from the lecture notes of CSCI3180, prepared by Lauren Pick, as well as some online resources.
This note is intended solely as a study aid. While I have done my best to ensure the accuracy of the content, I do not take responsibility for any errors or inaccuracies that may be present. Please use the material thoughtfully and at your own discretion.
If you believe any part of this content infringes on copyright, feel free to contact me, and I will address it promptly.
Mistakes might be found. So please feel free to point out any mistakes.

Contents

1 Foundations **2**
1.1 Syntax and Semantics 2

Chapter 1

Foundations

1.1 Syntax and Semantics

Consider an arbitrary programming language $\mathcal{L}$ (a set of programs). We have:

- **Syntax**: describes which things are programs in $\mathcal{L}$

- **Semantics**: describes what happens when we run a program in $\mathcal{L}$

So, given the syntax for a programming language $\mathcal{L}$, with a set of values $\mathcal{V}$, we define

$$\mathsf{Interpreter}_{\mathcal{L}} : \mathcal{L} \to \mathcal{V}$$

Here the *interpreter* takes a program in $\mathcal{L}$ and runs it to produce a value in $\mathcal{V}$. For example,

$$\mathsf{Interpreter}_{\text{Python}}(\texttt{sorted([1, 4, 3, 2])}) = \texttt{[1, 2, 3, 4]}$$

This can be seen as one way of defining the semantics of the program.

A compiler translates from one language to another. For a program p in $\mathcal{L}_1$ and $\mathsf{Compiler} : \mathcal{L}_1 \to \mathcal{L}_2$ to be correct, we want

$$\mathsf{Interpreter}_{\mathcal{L}_1}(p) = \mathsf{Interpreter}_{\mathcal{L}_2}(\mathsf{Compiler}(p))$$

otherwise something is incorrect.

Notice that interpreters and compilers, in **theory**, are mathematical objects, but not in reality. They have implementations, and while we can treat implementations of programming languages as defining their semantics, it is helpful not to do this, since implementations of interpreters and compilers are themselves programs and can be buggy.

Furthermore, different implementations of the same programming language may exhibit *diverging behaviors*, especially in the presence of under-specification. We can see that it is helpful to know what programs are meant to do (i.e., their semantics) separately from their implementations.